



AURA.COST

AI-POWERED INDUSTRIAL COST OPTIMIZATION ASSISTANT

Optimize Today. *Save Tomorrow.*

A closed-loop AI assistant that detects cost-relevant equipment anomalies, explains their root cause, quantifies the savings opportunity, recommends a corrective action, and tracks whether the saving was actually realized.

PROBLEM STATEMENT ID

[Add PS ID]

THEME

[Add Theme]

TEAM ID

[Add Team ID]

PROBLEM STATEMENT TITLE

AURA.COST — AI-Powered Industrial Cost Optimization Assistant

PS CATEGORY

Software

TEAM NAME

[Add Team Name]



IDEA TITLE

THE PROBLEM

- ✦ Plant costs (energy, material, downtime) are tracked in separate systems — ERP/Finance, MES/Production, IoT/Sensors, Maintenance/CMMS — reviewed on different cadences.
- ✦ Conventional monitoring is reactive and threshold-based: it flags a reading but does not explain why it happened.
- ✦ Root-cause investigation today is manual and slow, depending on institutional memory rather than data.
- ✦ Fixes are rarely tied to a quantified, trackable saving — and almost never followed up to confirm it was real.

PROPOSED SOLUTION — AURA.COST

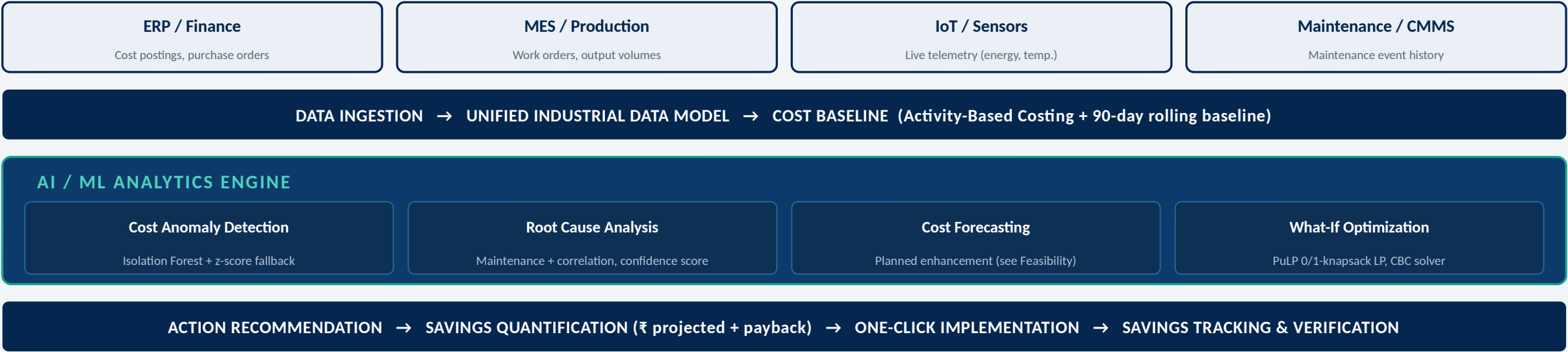
- ✦ A single continuously-running app that ingests machine telemetry, detects cost-relevant anomalies, and automatically explains their likely root cause.
- ✦ Every explained anomaly gets a recommended action with a projected rupee saving — implementable in one click from the dashboard, root-cause page, What-If sandbox, or an AI chat assistant.
- ✦ The loop closes: implemented actions are tracked through a verification window and compared, projected vs. realised.
- ✦ A Gemini-powered Co-Pilot answers questions in plain language and can trigger the same Implement action — one source of truth with the dashboard.

HOW IT WORKS — THE CLOSED LOOP





TECHNICAL APPROACH



🔄 Continuous Learning — verified action outcomes (Action Tracker) feed back into cost baselines and the AI/ML analytics engine.

TECHNOLOGY STACK (AS BUILT)

FRONTEND	BACKEND	AI & OPTIMIZATION	REALTIME & INFRA
✦ React 18.3 + TypeScript, Vite 5.4 ✦ Tailwind CSS 3.4 ✦ Zustand 5.0 (state) ✦ Recharts 2.13 (charts)	✦ FastAPI 0.115 (Python 3.11) ✦ SQLAlchemy 2.0 ORM ✦ Pydantic 2.9 (validation) ✦ PostgreSQL 16	✦ scikit-learn — Isolation Forest ✦ PuLP 2.9 (CBC solver) ✦ Google Gemini gemini-2.5-flash ✦ Function-calling Co-Pilot + fallback	✦ APScheduler 3.10 (30s tick) ✦ Native WebSockets (/ws/live, /ws/chat) ✦ PBKDF2-HMAC-SHA256 auth ✦ Docker Compose + nginx



FEASIBILITY AND VIABILITY

Technical Feasibility

- ✦ The full closed loop (detect → explain → recommend → implement → verify) runs end-to-end today.
- ✦ Verified against a real, running PostgreSQL database and real browser sessions — not a mock-up.
- ✦ Built entirely on a proven, mainstream open-source stack.

Operational Feasibility

- ✦ For a real plant, the missing pieces are a genuine telemetry connector and API-level authorization.
- ✦ Both are scoped, localized changes, not redesigns — the code already marks where a real connector plugs in.

Economic Viability

- ✦ Every component is free/open-source or a free API tier — confirmed line-by-line across the stack.
- ✦ \$0 running cost today; no paid cloud service is called anywhere in the current implementation.

Scalability

- ✦ Current design (single Postgres instance, in-process WebSocket manager) fits a single-plant prototype.
- ✦ Scaling to multiple plants/users needs a pub/sub layer, persisted models, and connection-pooling review — already scoped as near-term work.

CHALLENGES → MITIGATION

No API-level authorization — every backend endpoint is reachable without credentials today.	→ Add session/JWT-checked endpoints (a JWT_SECRET setting already exists, currently unused) before any real deployment.
Two frontend pages (What-If, Reports) are not yet wired to the live backend.	→ Wire What-If to the already-working, independently-verified api.runWhatIf(); add a backend aggregation endpoint for Reports.
No automatic savings-verification job — pending → verified is currently demo-seeded only.	→ Add a scheduled job that re-measures fct_cost / fct_sensor_reading after the verification window and closes the loop automatically.
Single plant, two machines — the data model currently hard-depends on this scope.	→ Generalize the machine-count assumptions in the join logic; extend the data model toward multi-plant support.

Validation scope: Tested locally against a real PostgreSQL database and browser sessions — 1 simulated plant, 2 machines, 36,920 sensor readings, 14 anomalies, 12 recommendations (5 implemented), over a 12-month seeded dataset. Not yet validated on real plant data or a production deployment.



IMPACT AND BENEFITS

KEY BENEFITS

- ✦ Continuous cost visibility — one dashboard instead of a periodic report.
- ✦ Early anomaly detection — every new reading scored in near real time (30s cadence).
- ✦ Explainable root causes — ranked driver + confidence score, drill-down to the machine.
- ✦ Quantified savings — every recommendation ships with a projected saving & payback.
- ✦ One-click corrective action from the dashboard, root-cause page, What-If, or AI Co-Pilot.
- ✦ Closed-loop verification — realised savings tracked against what was projected.

WHO BENEFITS

Plant Operations Managers

See the live alert the moment a cost anomaly appears, with a likely cause already surfaced.

Maintenance Engineers

Get an automatically-ranked explanation instead of starting a manual investigation.

Production / Cost Analysts

Review the Action Tracker to see which past fixes actually delivered their projected savings.

Management / Leadership

Ask the AI Co-Pilot a plain-language question, backed by the same live dashboard numbers.

TRADITIONAL VS. AURA.COST

Dimension	Traditional / Status Quo	AURA.COST
Cadence	Periodic (weekly/monthly) reporting, reconciled after the fact	Continuous, near real-time (30-second telemetry ticks)
Data	Siloed across ERP / MES / IoT / CMMS	Unified industrial data model across all sources
Root cause	Manual investigation, institutional memory	Automatic, ranked driver + confidence score
Action	Ad-hoc, rarely quantified	Recommended action, projected saving & payback, one click
Verification	Rarely followed up	Closed loop — tracked, projected vs. realised

SAMPLE SYSTEM OUTPUT — an actual recommendation record generated by the prototype

ASSET	ACTION	PROJECTED SAVINGS	IMPLEMENTATION COST	PAYBACK	CONFIDENCE
Extruder 3A	Service cooling jacket circulation pump	₹1,76,000	₹6,000	1 day	79%

Figures above are a real sample output of the working system — not a claim of verified real-plant results (see Feasibility & Viability).

RESEARCH AND REFERENCES

EXISTING LANDSCAPE

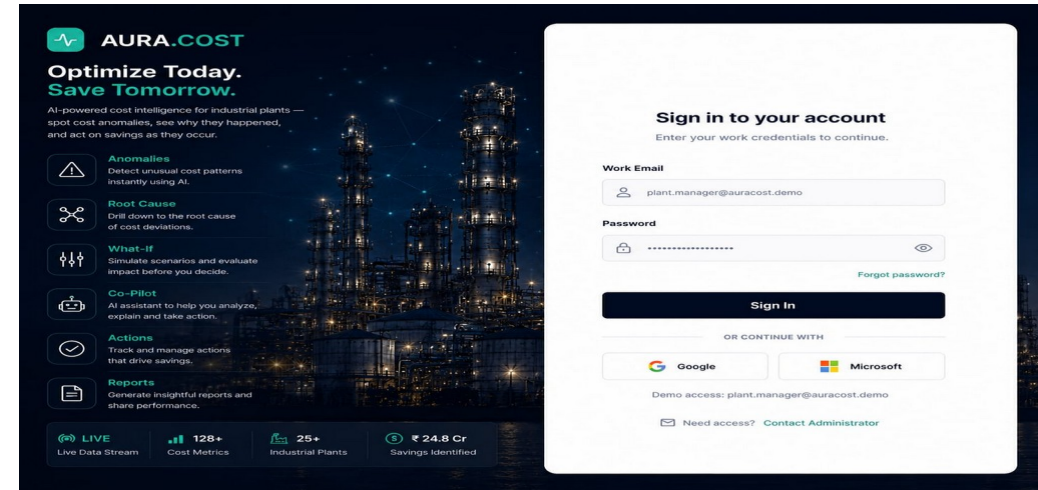
- ✦ Generic industry practice: cost variance is typically reviewed on a periodic (weekly/monthly) cadence — spreadsheets or a BI dashboard reconciled after the fact against ERP postings.
- ✦ Condition-monitoring / predictive-maintenance tools, as a general product category, typically focus on equipment health and failure prediction — not on translating an anomaly directly into a cost figure and a trackable action.
- ✦ Gap identified: the disconnect between real-time operational telemetry and financial cost visibility, and the absence of a closed verification loop on recommended actions.

No vendor-specific competitive audit was performed for this prototype — this section reflects the general, commonly understood landscape only.

HOW AURA.COST IS DIFFERENT

- ✦ Genuinely closed-loop, not just alerting — carries an anomaly through explanation, a quantified action, one-click implementation from four UI surfaces, and a tracked verification record.
- ✦ Chat and dashboard share one source of truth — the AI Co-Pilot's tools call the exact same backend functions as the REST API, so they can never structurally disagree.
- ✦ Zero running cost — every layer is free/open-source or a free tier, and the app degrades gracefully (not to a broken state) without a Gemini key.
- ✦ Honest architecture — genuine general-purpose techniques (the PuLP linear program, PBKDF2 hashing) are explicitly distinguished from illustrative heuristics standing in for a fuller build.

PROTOTYPE EVIDENCE



Actual login screen of the working AURA.COST prototype — real product logo, tagline, and module list (Anomalies, Root Cause, What-If, Co-Pilot, Actions, Reports).

REFERENCES

- ✦ FastAPI — fastapi.tiangolo.com
- ✦ React — react.dev
- ✦ PostgreSQL — postgresql.org/docs
- ✦ scikit-learn (Isolation Forest) — scikit-learn.org
- ✦ PuLP — coin-or.github.io/pulp
- ✦ Google Gemini API — ai.google.dev/gemini-api/docs

Only sources actually applicable to this project are listed — no fabricated citations or market studies.